



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Algorithm Analysis Fundamentals

Time complexity analysis of iterative and recursive algorithms; verify BigO, Ω , Θ for sample programs

NAME: Ganji Madhan Kumar

REG NO: 192472328

Experiment 1: Linear Search (Iterative)

The screenshot displays the Programiz Python Online Compiler interface. On the left, a file explorer shows 'main.py'. The main editor contains the following Python code:

```
1 arr = [10, 25, 30, 45, 50]
2 key = 30
3 found = False
4 for i in range(len(arr)):
5     if arr[i] == key:
6         print("Key found at index", i)
7         found = True
8         break
9
10 if not found:
11     print("Key not found")
```

On the right, the 'Output' pane shows the result of the execution:

```
Key found at index 2
=== Code Execution Successful ===
```

Buttons for 'Run', 'Share', and 'Clear' are visible at the top of the editor and output sections.

Experiment 2: Binary Search (Recursive)


Python Online Compiler

Programiz PRO >

main.py

```
1- def binary_search(arr, low, high, key):
2-     if low > high:
3-         return -1
4-
5-     mid = (low + high) // 2
6-
7-     if arr[mid] == key:
8-         return mid
9-     elif key < arr[mid]:
10-        return binary_search(arr, low, mid - 1, key)
11-     else:
12-        return binary_search(arr, mid + 1, high, key)
13-
14- arr = [5, 10, 15, 20, 25]
15- key = 20
16-
17- result = binary_search(arr, 0, len(arr) - 1, key)
18-
19- if result != -1:
20-     print("Key found at index", result)
21- else:
22-     print("Key not found")
```

Output

Clear

Key found at index 3
=== Code Execution Successful ===

Experiment 3: Factorial (Iterative and Recursive)


Python Online Compiler

Programiz PRO >

main.py

```
1 n = 5
2
3 # Iterative
4 fact = 1
5- for i in range(1, n + 1):
6-     fact *= i
7-
8 print("Iterative Factorial =", fact)
9
10 # Recursive
11- def factorial(n):
12-     if n == 0 or n == 1:
13-         return 1
14-     return n * factorial(n - 1)
15-
16 print("Recursive Factorial =", factorial(n))
```

Output

Clear

Iterative Factorial = 120
Recursive Factorial = 120
=== Code Execution Successful ===

Experiment 4: Fibonacci Series


Python Online Compiler

Programiz PRO >



main.py



```
1 n = 6
2
3 a = 0
4 b = 1
5
6 for i in range(n):
7     print(a, end=" ")
8     c = a + b
9     a = b
10    b = c
```

Output

Clear

```
0 1 1 2 3 5
=== Code Execution Successful ===
```

Experiment 5: Matrix Multiplication


Python Online Compiler

Programiz PRO >



main.py



```
1 A = [[1, 2],
2       [3, 4]]
3
4 B = [[5, 6],
5       [7, 8]]
6
7 result = [[0, 0], [0, 0]]
8
9 for i in range(2):
10     for j in range(2):
11         for k in range(2):
12             result[i][j] += A[i][k] * B[k][j]
13
14 print(result)
```

Output

Clear

```
[[19, 22], [43, 50]]
=== Code Execution Successful ===
```

Experiment 6: Merge Sort

Programiz
Python Online Compiler

Programiz PRO

main.py

```
1 def merge_sort(arr):
2     if len(arr) > 1:
3
4         mid = len(arr) // 2
5         left = arr[:mid]
6         right = arr[mid:]
7
8         merge_sort(left)
9         merge_sort(right)
10
11         i = j = k = 0
12
13         while i < len(left) and j < len(right):
14             if left[i] <= right[j]:
15                 arr[k] = left[i]
16                 i += 1
17             else:
18                 arr[k] = right[j]
19                 j += 1
20                 k += 1
21
22         while i < len(left):
23             arr[k] = left[i]
24             i += 1
25             k += 1
26
27         while j < len(right):
28             arr[k] = right[j]
29             j += 1
30             k += 1
```

Output

Original Array: [38, 27, 43, 3, 9, 82, 10]
Sorted Array: [3, 9, 10, 27, 38, 43, 82]

=== Code Execution Successful ===

Experiment 7: Quick Sort

Programiz
Python Online Compiler

main.py

```
1 # Quick Sort
2
3 def quick_sort(arr):
4     if len(arr) <= 1:
5         return arr
6
7     pivot = arr[len(arr) // 2]
8
9     left = [x for x in arr if x < pivot]
10    middle = [x for x in arr if x == pivot]
11    right = [x for x in arr if x > pivot]
12
13    return left + middle + right
14
15 arr = [10, 7, 8, 9, 1, 5]
16
17 print("Sorted Array:", quick_sort(arr))
```

Output

Sorted Array: [7, 8, 1, 5, 9, 10]

=== Code Execution Successful ===

Experiment 8: Tower of Hanoi

Programiz
Python Online Compiler

main.py

```
1 # Tower of Hanoi
2
3 def hanoi(n, source, auxiliary, destination):
4     if n == 1:
5         print("Move disk 1 from", source, "to", destination)
6         return
7
8     hanoi(n - 1, source, destination, auxiliary)
9     print("Move disk", n, "from", source, "to", destination)
10    hanoi(n - 1, auxiliary, source, destination)
11
12    n = 3
13
14    hanoi(n, 'A', 'B', 'C')
```

Output

Move disk 1 from A to C
Move disk 2 from A to B
Move disk 1 from C to B
Move disk 3 from A to C
Move disk 1 from B to A
Move disk 2 from B to C
Move disk 1 from A to C

=== Code Execution Successful ===

Experiment 9: GCD (Euclidean Algorithm)

Programiz
Python Online Compiler

```
main.py
1 # GCD using Euclidean Algorithm
2
3 def gcd(a, b):
4     if b == 0:
5         return a
6     return gcd(b, a % b)
7
8 a = 48
9 b = 18
10
11 print("GCD =", gcd(a, b))
```

Output

GCD = 6

=== Code Execution Successful ===

Experiment 10: Insertion Sort

Programiz
Python Online Compiler

```
main.py
1 # Insertion Sort
2
3 arr = [12, 11, 13, 5, 6]
4
5 for i in range(1, len(arr)):
6     key = arr[i]
7     j = i - 1
8
9     while j >= 0 and arr[j] > key:
10         arr[j + 1] = arr[j]
11         j -= 1
12
13     arr[j + 1] = key
14
15 print("Sorted Array:", arr)
```

Output

Sorted Array: [5, 6, 11, 12, 13]

=== Code Execution Successful ===

Experiment 11: Heap Sort

Programiz
Python Online Compiler

```
main.py
1 # Heap Sort
2
3 import heapq
4
5 arr = [4, 10, 3, 5, 1]
6
7 heapq.heapify(arr)
8
9 sorted_list = []
10
11 while arr:
12     sorted_list.append(heapq.heappop(arr))
13
14 print("Sorted Array:", sorted_list)
```

Output

Sorted Array: [1, 3, 4, 5, 10]

=== Code Execution Successful ===

Experiment 12: Counting Inversions

main.py	Output
<pre>1 # Counting Inversions (Brute Force) 2 3 arr = [2, 4, 1, 3, 5] 4 5 count = 0 6 7- for i in range(len(arr)): 8- for j in range(i + 1, len(arr)): 9- if arr[i] > arr[j]: 10- count += 1 11 12 print("Number of Inversions:", count)</pre>	Number of Inversions: 3 === Code Execution Successful ===

Experiment 13: Maximum Subarray Sum (Kadane's Algorithm)

main.py	Output
<pre>1 # Maximum Subarray Sum 2 3 arr = [-2, -3, 4, -1, -2, 1, 5, -3] 4 5 max_sum = arr[0] 6 current_sum = arr[0] 7 8- for i in range(1, len(arr)): 9 current_sum = max(arr[i], current_sum + arr[i]) 10 max_sum = max(max_sum, current_sum) 11 12 print("Maximum Subarray Sum =", max_sum)</pre>	Maximum Subarray Sum = 7 === Code Execution Successful ===

Experiment 14: Exponentiation (Fast Power)

main.py	Output
<pre>1 # Fast Exponentiation 2 3- def power(x, n): 4- if n == 0: 5- return 1 6 7 half = power(x, n // 2) 8 9- if n % 2 == 0: 10- return half * half 11- else: 12- return x * half * half 13 14 x = 2 15 n = 10 16 17 print("Result =", power(x, n))</pre>	Result = 1024 === Code Execution Successful ===

Experiment 15: Closest Pair of Points (Brute Force)



Python Online Compiler

main.py

```
1 # Closest Pair of Points
2
3 import math
4
5 points = [(1, 2), (4, 5), (7, 8), (3, 1)]
6
7 min_distance = float('inf')
8 closest_pair = ()
9
10 for i in range(len(points)):
11     for j in range(i + 1, len(points)):
12         x1, y1 = points[i]
13         x2, y2 = points[j]
14
15         distance = math.sqrt((x2 - x1) ** 2 + (y2 - y1) ** 2)
16
17         if distance < min_distance:
18             min_distance = distance
19             closest_pair = (points[i], points[j])
20
21 print("Closest Pair:", closest_pair)
22 print("Distance =", min_distance)
```

Run

Output

Closest Pair: ((1, 2), (3, 1))
Distance = 2.23606797749979

=== Code Execution Successful ===